PROJEKTPRÄSENTATION · TEAM CMSLPD

SIMS

Security Incident Management System

Sicherheitsvorfälle erfassen, SBOMs automatisch prüfen,
Tickets eskalieren – komplett in Docker.



„Sul Sul, Exploit!“ · Version 1.0.0 · github.com/r4nnd0m/ac_ProjectSeks

ABLAUF · 15 MINUTEN

Sechs Rollen, ein System

Teil	Rolle	Pakete	Zeit
Idee und Architektur	6 · [Name]	F1–F5	2:00
Anmeldung, MFA, Token	1 · [Name]	A1, A2, A4, A5	2:00
Scans, Datenmodell, Security	2 · [Name]	B1–B3, B5, S1	2:30
Tickets und Audit	3 · [Name]	C1–C3, C5	2:00
Worker	4 · [Name]	D1–D4	2:00
Urteil und Eskalation	6 · [Name]	B4, C4, D5	1:30
Frontend, News, Live-Demo	5 · [Name]	E1–E5, N1–N3, A3	2:30
SAST, Roadmap, Fragen	2 und 5	–	0:30

DIE IDEE

Zwei Wege zu einem Sicherheitsvorfall

MANUELL

Jemand meldet einen Vorfall

Titel, Beschreibung, Schweregrad, CVE, betroffenes System, Melder und Zeitstempel – direkt im Web-Frontend erfasst.

AUTOMATISCH

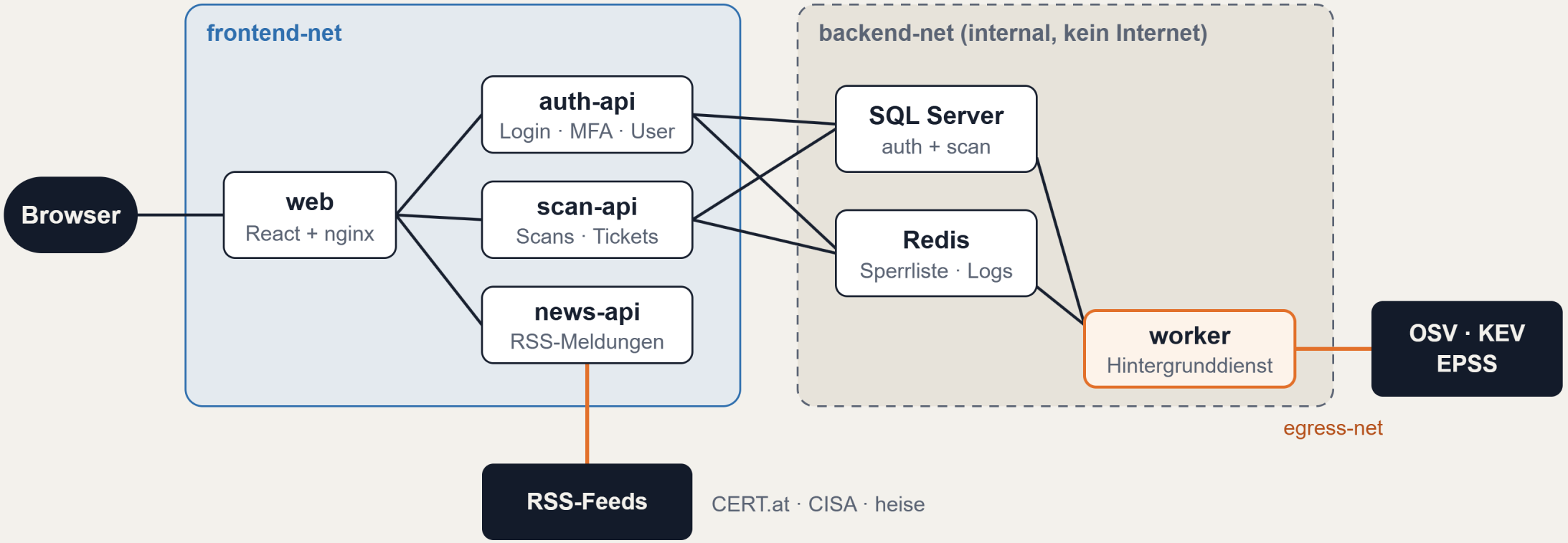
Ein Team lädt seine SBOM hoch

Der Worker prüft jede Komponente gegen OSV, CISA KEV und FIRST EPSS, fällt ein Urteil und legt Tickets selbst an.

Danach gilt für beide dasselbe: übernehmen, bearbeiten, schließen – oder automatisch eskalieren, wenn die Frist abläuft. Jede Aktion landet im Audit-Log.

ARCHITEKTUR

7 Container, 3 getrennte Netze



Ports nur auf 127.0.0.1 · Container ohne Root, cap_drop ALL, no-new-privileges

ROLLE 1 · AUTH-API

Anmeldung in zwei Schritten

01	02	03	04	05
Login	**Prüfung**	**MFA aktiv?**	**TOTP-Code**	**JWT**
E-Mail + Passwort an /auth/login	Aktiv? Gesperrt? Passwort-Hash korrekt?	Dann nur ein kurzes Zwischentoken	6-stelliger Code an /auth/mfa/verify	15 Minuten gültig, mit Rolle und Team

PASSWÖRTER

ASP.NET Core Identity, Hash mit Salt, mindestens 12 Zeichen

BRUTE FORCE

Sperre nach 5 Fehlversuchen für 15 min, Rate Limit 5 pro Minute und IP

MFA EINRICHTEN

QR-Code im Frontend, Bestätigung mit dem ersten Code

ROLLE 1 · AUTH-API

Token, Rollen und Abmelden

IM JWT

`sub · name · role`
`team_id · mfa · jti`

Die Scan-API prüft das Token selbst – Aussteller, Zielgruppe, Signatur, Laufzeit.

DREI ROLLEN

Developer – arbeitet Tickets ab
Lead – bekommt Eskalationen
Admin – verwaltet User, Teams, Audit

LOGOUT

Die Token-ID kommt in eine Redis-Sperrliste, mit Ablaufzeit = Restlaufzeit des Tokens.

Ist Redis weg, wird abgelehnt: **fail closed**.

Kritische Aktionen wie „Risiko akzeptieren“ verlangen Admin **und** ein Token mit bestätigter MFA.

ROLLE 2 · SCAN-API A

Projekte und SBOM-Upload

PROJEKT

Gehört einem Team

Jedes Projekt hat eine TeamId. Wer nicht im Team ist, bekommt 404 – nicht einmal die Existenz wird verraten.

UPLOAD

CycloneDX-JSON

Mit Versionsangabe. Höchstens 5 MB und 5 000 Komponenten. Antwort: 202 Accepted, Scan steht auf Pending.

VALIDIERUNG

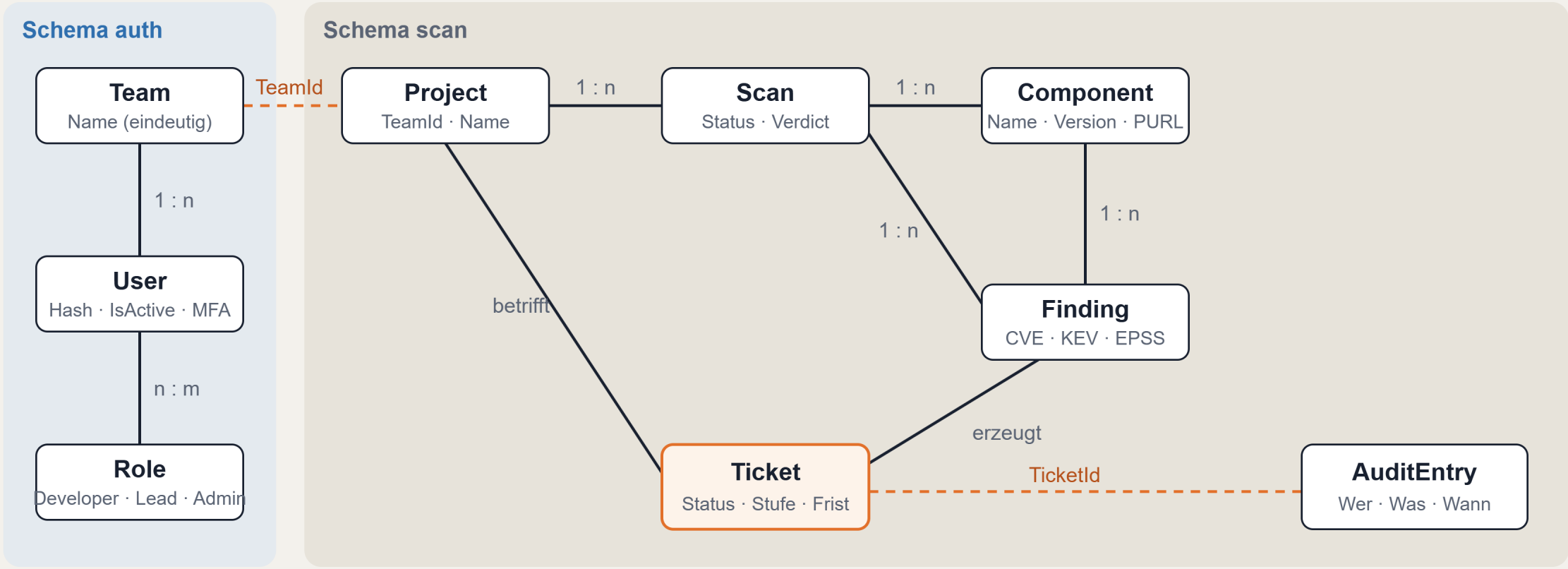
Alle Fehler auf einmal

Format, Name, Version, gültige PURL, Duplikate. Fehler gesammelt als HTTP 400 statt Serverfehler.

```
POST /projects/{id}/scans?version=1.0.0 → 202 { scanId, "Pending" }
```

ROLLE 2 · DATENMODELL

Eine Datenbank, zwei Schemas



Entity Framework Core · Code First mit Migrationen · keine selbst gebauten SQL-Strings

ROLLE 2 · SECURITY-BASIS

Sicherheit in jeder Schicht

NETZ

SQL und Redis im internen Netz, Ports nur auf 127.0.0.1

CONTAINER

Ohne Root, alle Capabilities entzogen, Multi-Stage-Builds ohne SDK

HTTP

CSP, nosniff, X-Frame-Options DENY, CORS nur für eigene Ursprünge

LAST

Rate Limit pro IP: 100 pro Minute, Login 5 pro Minute

EINGABEN

DTO-Validierung, Größenlimits, nur EF Core – kein SQL-String

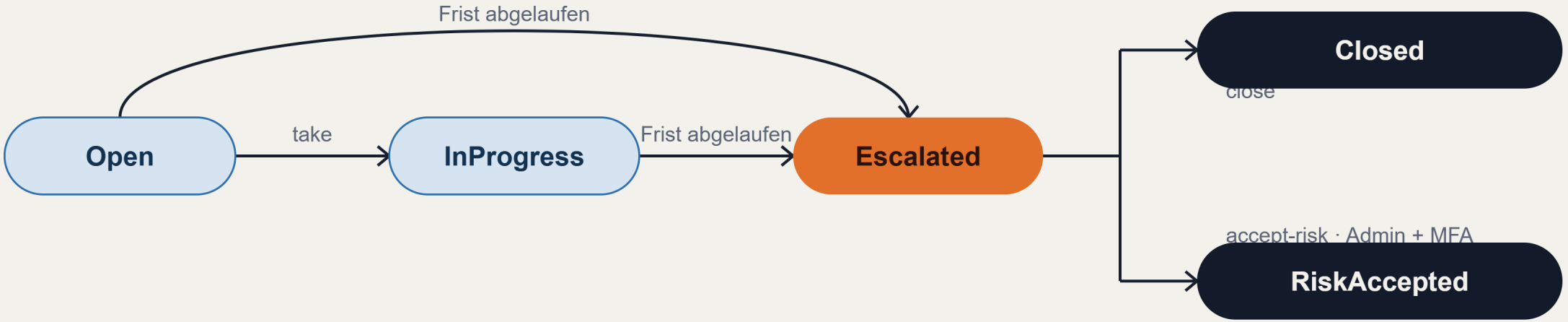
GEHEIMNISSE

Nur in .env, nie im Code. Fehlt der JWT-Schlüssel, startet nichts

Vollständige Liste in der README unter „Security-Maßnahmen“

ROLLE 3 · SCAN-API B

Der Lebenszyklus eines Tickets



Schließen und Risiko akzeptieren gehen aus jedem offenen Status.

Die Klasse Ticket schützt ihre Regeln selbst: private Setter, Zustandswechsel nur über Methoden, ein abgeschlossenes Ticket ist unveränderlich.

ROLLE 3 · SCAN-API B

Automatische Tickets und Audit-Log

C3 · AUTOMATISCH

Ein Ticket pro relevantem Fund

- Ab Schweregrad Medium oder bei KEV-Eintrag
- Bestehende Tickets werden nicht doppelt angelegt
- Startfrist kommt direkt aus der Eskalationsregel

C5 · AUDIT

Wer hat wann was getan?

- Übernommen, geschlossen, eskaliert, Risiko akzeptiert
- Mit Benutzer, Zeitpunkt und Ticket
- Nur für Admins einsehbar, seitenweise

Ticket-API	Wer darf
GET /tickets?status=&severity=&assignedToMe=	eigenes Team
POST /tickets · PUT /tickets/{id} · .../take · .../close	eigenes Team
POST /tickets/{id}/accept-risk · GET /audit	Admin (+ MFA)

ROLLE 4 · WORKER

Vom Upload zum Urteil

01	02	03	04	05	06
Abholen	**OSV**	**CISA KEV**	**FIRST EPSS**	**Urteil**	**Speichern**
Alle 30 s wartende Scans aus SQL, Status → Scanning	Bekannte Schwachstellen je Paket-URL	Wird die Lücke aktiv ausgenutzt?	Wahrscheinlichkeit eines Angriffs	Go · mit Auflagen · NoGo	Funde, Urteil und Tickets in einer Transaktion

OHNE HTTP-ENDPUNKT

Ein reiner .NET-Hintergrunddienst mit drei Jobs: Scans, Eskalation, Lebenszeichen.

EINZIGER WEG NACH AUSSEN

Nur der Worker hängt im egress-Netz – die Datenbanken bleiben abgeschottet.

ROLLE 4 · WORKER

Gebaut für den Ausfall

NEUSTART

Nichts bleibt hängen

Scans, die beim Absturz auf Scanning standen, setzt ScanRecovery wieder auf Pending.

QUELLE DOWN

Sauber gescheitert

Retry und Timeout pro Abfrage. Danach Status Failed – ohne Urteil, ohne falsche Tickets.

LOGGING

Zentral in Redis

Id, Timestamp, Level, Service, Message. Ohne sensible Fehlertexte, max. 10 000 Einträge.

73

Worker-Tests

mit HTTP-Stubs und Testuhr

3

externe Quellen

OSV · KEV · EPSS

0

Tickets bei Fehler

lieber kein Urteil als ein falsches

SIMS · Rolle 4 · D1, D4

ROLLE 6 · SCANURTEIL

Darf diese Version ausgeliefert werden?

Go

Nur niedrige Funde oder gar keine.

Go mit Auflagen

Mindestens ein mittlerer oder hoher Fund – Tickets müssen abgearbeitet werden.

NoGo

Ein KEV-Eintrag, ein kritischer Fund oder EPSS ≥ 0,5.

STRENGSTES GEWINNT

Drei unabhängige Regeln – KEV, CVSS, EPSS – liefern je ein Urteil, das schärfste zählt. Ein fehlender EPSS-Wert löst nichts aus.

ERWEITERBAR

Jede Regel ist eine eigene Klasse hinter IVerdictRule. Eine neue Regel braucht keine Änderung am Rest.

ROLLE 6 · ESKALATION

Niemand reagiert? Dann die nächste Stufe.

Stufe 1	Stufe 2	Stufe 3
Team	**Lead**	**Admin**
Critical 4 h · High 24 h · Medium 72 h · Low ohne Frist	Neue Frist gleicher Länge ab dem Prüfzeitpunkt	Endstation, keine weitere Frist

ABKÜRZUNG

KEV-Funde starten sofort auf Stufe 3 beim Admin.

AUTOMATISCH

Der EscalationJob im Worker prüft regelmäßig alle offenen Tickets.

NACHVOLLZIEHBAR

Jede Eskalation landet im Audit-Log. Erledigte Tickets bleiben unberührt.

ROLLE 5 · FRONTEND

Eine Oberfläche für alle drei APIs

ALLE

Dashboard

Offene und kritische Tickets, Projekte, letzte Scans

ALLE

Projekte und Scans

SBOM hochladen, Urteil und Fundliste ansehen

ALLE

Tickets

Filtern, übernehmen, schließen, neu erfassen

ALLE

MFA einrichten

QR-Code scannen, Code bestätigen

ADMIN · A3

Verwaltung

Benutzer anlegen und deaktivieren, Teams, Audit-Log

ÖFFENTLICH

Security-News

Aktuelle Warnungen ohne Login

React 18 · TypeScript · Vite · nginx · helles und dunkles Theme

ROLLE 5 · NEWS-SERVICE

Was gerade in der Security-Welt passiert

N1	N2	N3
Feeds lesen	**Cachen**	**Bereinigen**
CERT.at, CISA und heise Security – nur über https	Ergebnis 30 Minuten im Speicher, die Quellen werden geschont	HTML-Tags raus, Links nur http/https

Eigener kleiner Microservice ohne Datenbank und ohne Login – fällt ein Feed aus, liefern die anderen weiter.

ROLLE 5 · LIVE-DEMO · CA. 90 SEKUNDEN

Einmal durch das ganze System

1. Als Admin anmelden – mit MFA-Code aus der Authenticator-App
2. Projekt anlegen und test-sbom.json als Version 1.0.0 hochladen
3. Kurz warten: der Scan springt von Pending auf Scanned – Urteil **NoGo** wegen log4j
4. Automatisch erzeugte Tickets öffnen, eines übernehmen und schließen
5. Manuelles Ticket erfassen und das Risiko als Admin akzeptieren
6. Audit-Log zeigen – jede Aktion ist nachvollziehbar

Vorher: docker compose up -d --build · http://localhost:8080 · Plan B: Frontend im Demo-Modus

ROLLE 2 · CODE-QUALITÄT

Was die statische Analyse gefunden hat

0

Funde im Quellcode

Semgrep · 138 Regeln · 250 Dateien

0

anfällige NuGet-Pakete

11 Projekte, inklusive transitiver

2

mittlere npm-Befunde

react-router · in SIMS nicht ausnutzbar

Vor der Prüfung haben wir im Code-Review selbst vier Lücken geschlossen – unter anderem fehlende Team-Prüfung bei Scans und einen Fallback-Schlüssel im Code.

Bericht: Zusatz/SIMS_Sicherheitspruefung_2026-11-13.pdf · Rolle 2 · ca. 30 Sekunden

ROADMAP

Wie es weitergehen könnte

Version	Wann	Inhalt
1.0.0	Nov 2026	Heute: 3 APIs, Worker, Frontend, MFA, Eskalation, Audit, Docker, SAST
1.1.0	Dez 2026	react-router-Update, CI-Pipeline mit Build, Tests und Semgrep
1.2.0	Jän 2027	SBOM im Build und Dependency-Track als eigener Container
1.3.0	Feb 2027	Sofortiger Token-Widerruf beim Deaktivieren, Refresh-Tokens
1.4.0	Mär 2027	Benachrichtigung bei Eskalation per E-Mail oder Teams
2.0.0	Q2 2027	Jira-Anbindung, eigene SQL-Logins je Dienst, TLS, Cloud-Deployment

TEAM CMSLPD

Fragen?

7 Container · 3 Microservices + Worker · SQL Server + Redis · 133 automatisierte Tests

github.com/r4nnd0m/ac_ProjectSeks

